

Web APIs (Storage)

1. What is a Web API?

- a. **API** (Application Programming Interface): A set of functions that allows interaction between applications.
- b. **Web API**: The Web Storage API allows web applications to store small pieces of data in the browser using key-value pairs. It is a built-in API.
- c. **Examples**:
 - i. DOM API
 - ii. Fetch API
 - iii. Web Storage API
- d. Here, we focus on **Web Storage API** (managing data on the client side).

2. Ways to Store Data on Browser Side:

- a. We can store the data on client side means browser side in 3 ways:
 - i. By Using Local Storage
 - ii. By Using Session Storage
 - iii. By Using Cookies

3. Local Storage:

- a. Stores data *without any expiration time*.
- b. Data remains even after the browser is closed and reopened.
- c. Good for storing user preferences, theme settings, tokens, etc.

d. **Syntax and Methods:**

```
// Store data
window.localStorage.setItem("key", "value");

// Get data
let data = window.localStorage.getItem("key");

// Remove specific item
window.localStorage.removeItem("key");

// Clear all local storage
window.localStorage.clear();
```

4. **Session Storage:**

- a. Stores data *only for one session* (until the tab or browser is closed).
- b. Data is cleared when the session ends.
- c. Useful for temporary data like form inputs, session tokens.

d. **Syntax and Methods:**

```
// Store data
window.sessionStorage.setItem("key", "value");

// Get data
let data = window.sessionStorage.getItem("key");

// Remove specific item
window.sessionStorage.removeItem("key");

// Clear all session storage
window.sessionStorage.clear();
```

5. Key Differences: Local Storage vs Session Storage

Feature	Local Storage	Session Storage
Persistence	Data persists indefinitely (until explicitly deleted).	Data persists only for the duration of the browser session (until the tab or window is closed).
Storage Capacity	Approximately 5-10 MB, depending on the browser.	Roughly 5 MB, similar to Local Storage but may be lower depending on the browser.
Scope	Shared across all tabs and windows of the same origin (domain).	Scoped to the tab or window where it was created. Not shared between tabs.
Use Cases	Storing user preferences, theme settings, authentication tokens, etc.	Temporary data such as form data or per-session state (e.g., filters).
Access Method	Accessed via <code>localStorage</code> object in JavaScript.	Accessed via <code>sessionStorage</code> object in JavaScript.

6. Cookies:

- A cookie is a small piece of text data that a website stores on your computer or device when you visit it.
- Cookies help websites recognize you and provide a personalized experience based on your preferences.
- Cookies have been used on the internet for a long time to make web browsing more efficient.
- Mostly used for *authentication*, *tracking*, and *personalization*.

e. Examples of Cookie Usage:

- i. Remembering login credentials.
- ii. Keeping items in a shopping cart.
- iii. Showing personalized content, like recommendations or ads.

f. Syntax and Methods:

```
// Create a cookie
```

```
document.cookie = "user=John";
```

```
// Read cookies
```

```
console.log(document.cookie);
```

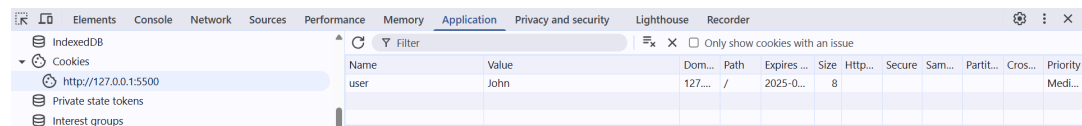
```
// Update cookie (set again with same name)
```

```
document.cookie = "user=Smith";
```

```
// Delete cookie (set expiry date to past)
```

```
document.cookie = "user=John; expires=Mon, 22  
Sept 2025 00:00:00 UTC; path=/;";
```

g. Output:



The screenshot shows the Chrome DevTools Application tab with the 'Cookies' section expanded. A table lists the cookies for the URL 'http://127.0.0.1:5500'. The table has columns for Name, Value, Dom..., Path, Expires..., Size, Http..., Secure, Sam..., Partit..., Cros..., and Priority. One cookie is listed with Name 'user' and Value 'John'.

Name	Value	Dom...	Path	Expires ...	Size	Http...	Secure	Sam...	Partit...	Cros...	Priority
user	John	127...	/	2025-0...	8						Medi...

7. Which One to Use?

- a. *Local Storage* → Use when you need to store data permanently on the client (e.g., theme, saved preferences, JWT token).
- b. *Session Storage* → Use for temporary data that should vanish once the session ends (e.g., unsaved form inputs, session token).
- c. *Cookies* → Use when data needs to be sent to the server with every request (e.g., authentication sessions, tracking user activity).